M15 TDD / BDD / SDD Workflows

Methodology Intermediate

Structured development methodologies with Claude Code

Comparing the three approaches

Aspect	TDD	BDD	SDD
Starting point	Technical test	Business behavior	Written specification
Language	Test code	Given/When/Then	Structured document
Audience	Developers	Devs + business	Entire team
Feedback	Tests pass/fail	Validated scenarios	Spec implemented

TDD with Claude: Red-green-refactor

The classic cycle, made explicit in prompts to prevent Claude from jumping straight to implementation.

```
# RED phase: ask for the failing test first
> "Write a FAILING test for isValidEmail().
  The function does not exist yet."

# GREEN phase: minimal implementation
> "Write the minimal code to make this test pass."

# REFACTOR phase: improvement
> "Improve the implementation. Tests must
  stay green."
```

Being explicit about “FAILING” and “does not exist yet” matters. Otherwise Claude tends to write test and implementation together.

BDD with Claude: Given/when/then

```
> "Describe the behavior of the login page
  as Gherkin scenarios, then implement."

# Claude generates:
# Scenario: Login with valid credentials
#   Given a registered user with email test@ex.com
#   When they submit the form with the correct password
#   Then they are redirected to their dashboard
```

Scenarios serve simultaneously as spec, documentation, and test base, in a language readable by non-developers.

SDD with Claude: Spec first

Write the complete specification before the first line of code. Claude then uses the spec as a reference throughout the implementation.

```
> "Here is the spec for POST /auth/login endpoint
  [paste the spec]. Implement section by section
  respecting every constraint."
```

Verification Loops: the founding principle

All three methods rely on the same principle identified by Boris Cherny: **an agent that can “see” what it has produced delivers better results.** Without a verification mechanism, Claude iterates blindly.

Domain	Verification tool
Backend	Tests (unit, integration)
Frontend	Live browser preview
Types	TypeScript compiler
Style	ESLint / Prettier
Security	Semgrep, static analyzers

Chain-of-verification

Cross-validation pattern (arXiv:2309.11495) to reduce hallucinations: Claude generates a response, then generates verification questions about that response, then answers those questions to detect inconsistencies.

Applicable when result correctness is critical and automated tests are insufficient (complex algorithms, subtle business logic).

Integration with tasks API

Enable task tools with `CLAUDE_CODE_ENABLE_TODO_TOOLS=1`, then ask Claude:

```
Create tasks for failing auth tests, implementation, and refactoring.
Use the IDs returned by TaskCreate to add dependencies with
TaskUpdate:
implementation waits for tests; refactoring waits for implementation.
```

The Red-Green-Refactor cycle becomes traceable and persistent across sessions.